

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 05 -

ESTRATÉGIAS DE DEPLOY AVANÇADAS: BLUE/GREEN

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	17
O QUE VOCÊ VIU NESTA AULA?	17
REFERÊNCIAS.....	23

O QUE VEM POR AÍ?

Implantar novas versões de aplicações críticas é um verdadeiro jogo de equilíbrio: qualquer erro minúsculo pode derrubar o sistema e custar caro em receita e reputação. Lidar com downtime planejado (as famigeradas janelas de manutenção) muitas vezes não é uma opção viável em negócios globais que operam 24/7, a pressão por atualizações rápidas entra em conflito com a necessidade de estabilidade.

É nesse cenário desafiador que a estratégia **Blue/Green se destaca**, prometendo implantações sem interrupções perceptíveis. Em vez de atualizar “ao vivo” e torcer para nada dar errado, o Blue/Green mantém **dois ambientes de produção isolados e idênticos**, alternando o tráfego entre eles de forma quase instantânea.

Nesta aula, vamos descobrir como essa abordagem representa um verdadeiro “santo graal” do DevOps: **segurança operacional máxima** durante deploys, com **reversão instantânea** caso algo saia do previsto. Ao longo da introdução, mergulharemos nos desafios clássicos de deploys tradicionais e veremos como o Blue/Green aborda cada um deles – reduzindo risco, eliminando downtime e oferecendo a tão sonhada tranquilidade nas madrugadas de release.

Prepare-se para repensar a forma como você enxerga a entrega de software, pois um mundo de deploys sem sustos e com controle total está prestes a se descortinar.

HANDS ON

Chegou a hora de transformar teoria em prática. Imagine que temos uma aplicação containerizada e queremos realizar um deploy Blue/Green no Kubernetes, usando código em **Rust** para orquestrar tudo. Nesta atividade hands-on, vamos criar dois conjuntos de Pods – um representando a versão atual (ambiente **Blue**) e outro a nova versão (ambiente **Green**).

Em seguida, utilizaremos um **Service do Kubernetes** para alternar o tráfego entre essas versões, aproveitando a mágica dos labels (rótulos) e selectors (seletores) que o Kubernetes nos oferece.

Por fim, veremos como preparar um rollback instantâneo: com um simples comando via API, o tráfego volta para o ambiente antigo em segundos, comprovando o poder dessa estratégia. Tudo isso será feito utilizando a biblioteca kube-rs (cliente Kubernetes em Rust), demonstrando como interagir programaticamente com o cluster para criar recursos e efetuar a troca de forma controlada.

SAIBA MAIS

Vamos começar configurando o acesso ao cluster e criando os pods **Blue e Green**.

No código a seguir, utilizamos a crate kube para nos conectar ao cluster padrão, e a crate k8s_openapi para definir objetos Kubernetes. Primeiro, construímos um Pod da versão blue (v1.0 do nosso app) usando rótulos `app=myapp`, `env=blue`. Em seguida, procedemos de forma similar para a versão green (v2.0). Repare que definimos explicitamente os labels de ambiente em cada Pod – é assim que diferenciaremos os conjuntos:

```
usekube::{Client, Api};
usek8s_openapi::api::core::v1::Pod;
useserde_json::json;
usekube::api::PostParams;

#[tokio::main]
async fn main() -> Result<(), Box
```

Figura 1 – Criando pods Blue e Green
Fonte: elaborado pelo autor (2025)

Com os pods das duas versões já configurados (inicialmente podemos escalar o green para 0 réplicas até o momento do teste, se preferir, para economizar recursos), precisamos expor o serviço aos usuários.

No Kubernetes, um **Service** atua como ponto de acesso estável, responsável por rotear o tráfego para os Pods corretos com base em seus **rótulos**. Vamos criar um Service chamado `myapp-service` inicialmente apontando para a versão Blue. Para isso, definimos o selector do Service como `app=myapp`, `env=blue`:

```
usek8s_openapi::api::core::v1::Service;
usekube::api::ObjectMeta;

letservices:Api<Service>=Api::default_namespaced(client.clone());

letservice_blue=Service {
  metadata:ObjectMeta {
    name:Some("myapp-service".into()),
    ..Default::default()
  },
  spec:Some(k8s_openapi::api::core::v1::ServiceSpec {
    selector:Some([("app".into(), "myapp".into()), ("env".into(),
"blue".into())])
                    .iter().cloned().collect()),
    ports:Some(vec![k8s_openapi::api::core::v1::ServicePort {
      port:80,
      target_port:Some(IntOrString::Int(80)),
      protocol:Some("TCP".into()),
      ..Default::default()
    }]),
    type_:Some("ClusterIP".into()),
    ..Default::default()
  }),
  ..Default::default()
};
services.create(&PostParams::default(), &service_blue).await?;
println!("Service myapp-service criado apontando para BLUE");
```

Figura 2 – Service myapp-service
Fonte: elaborado pelo autor (2025)

Assim, qualquer requisição enviada ao `myapp-service` (porta 80) será roteada, por meio do kube-proxy para os pods com label `env=blue` – ou seja, para a versão

antiga da aplicação. Agora vem a troca Blue→Green, o momento crítico do Blue/Green deployment.

No Rust, podemos simplesmente patch (atualizar parcialmente) o objeto Service, alterando seu selector para apontar para o label env=green. Esse patch pode ser feito via JSON, alterando apenas o campo desejado:

```
use kube::api::{Patch, PatchParams};
use serde_json::json;

let patch = json!({
    "spec": { "selector": { "app": "myapp", "env": "green" } }
});
services.patch("myapp-service", &PatchParams::default(),
    &Patch::Merge(&patch)).await?;
println!("Service myapp-service atualizado para apontar para GREEN!");
```

Figura 3 – Patch Service
Fonte: elaborado pelo autor (2025)

Com essa atualização, o Kubernetes rapidamente **reconfigura o roteamento**: em poucos instantes, todas as novas conexões passam a ser direcionadas para os pods da versão Green (v2.0).

Parabéns, você acabou de promover a nova versão sem nenhum downtime!

Podemos validar o resultado listando os endpoints do Service ou verificando via `kubectl` que os pods verdes estão recebendo tráfego.

E se algo der errado com a versão Green?

Nesse caso, o rollback é tão simples quanto fazer o patch inverso – basta ajustar o selector de volta para env=blue. Como ainda mantivemos os pods antigos em execução e prontos, a reversão ocorre em segundos, restabelecendo o estado anterior instantaneamente. Essa rapidez de rollback é a essência da segurança operacional do Blue/Green: problemas críticos podem ser mitigados com um flip de configuração, sem necessidade de reimplantar a versão anterior (que já estava lá, quietinha, apenas sem tráfego).

E a teoria?

Até aqui, vimos como fazer, agora entenderemos por que o Blue/Green funciona tão bem, e quando ele realmente brilha, com um olhar mais **teórico e rigoroso**. Vamos dissecar formalmente o problema da **comutação segura de versões em produção** e examinar a estratégia Blue/Green por vários ângulos: consistência de dados, algoritmos de promoção/rollback, custos computacionais, modelagem matemática de risco e muito mais.

Formulação do problema: no deploy Blue/Green temos dois estados globais possíveis do sistema:

- Estado **B** (versão azul ativa);
- Estado **G** (versão verde ativa).

A comutação nada mais é do que uma transição controlada de B para G, idealmente atômica (do ponto de vista do usuário) e reversível. Formalmente, podemos pensar em uma função de roteamento $R(t)$ que, a cada instante, mapeia usuários para um ambiente.

No Blue/Green perfeito, $R(t)$ muda de 100% Blue para 100% Green em um intervalo de tempo tão curto que nenhum usuário percebe indisponibilidade. Essa troca deve satisfazer propriedades de consistência: todas as requisições devem ser atendidas ou pelo ambiente antigo ou pelo novo, nunca perdidas no limbo. Mais ainda, a estratégia exige **estado consistente** ou compatível entre as duas versões no momento do cutover, para que a experiência do usuário não seja impactada (imagine um usuário logado não precisar logar de novo após a troca, ou um pedido de e-commerce não se perder). Em termos de teoria de transações, buscamos algo próximo de uma **transição atômica** entre duas versões de serviço em produção.

CONSISTÊNCIA DE DADOS VS. COMPATIBILIDADE DE ESQUEMA

Aqui reside um dos maiores desafios práticos do Blue/Green. Manter duas versões rodando lado a lado é relativamente simples, quando ambas utilizam exatamente o mesmo modelo de dados. Porém, o cenário complica, quando a nova

versão requer **mudanças no banco de dados** ou em **contratos de APIs compartilhadas**

Nesse caso, a implantação Blue/Green exige preparação extra para garantir **compatibilidade retroativa** (backward compatibility) entre versões. Uma abordagem recomendada é **separar as mudanças de esquema das mudanças de código**, introduzindo compatibilidade dupla temporariamente. Martin Fowler sugere, por exemplo, aplicar primeiro migrações no banco que suportem simultaneamente a versão antiga e a nova, validar em produção e só então ativar o novo código usando esse esquema estendido.

Assim, caso precisemos reverter, a versão anterior ainda encontra o banco num estado utilizável. Sem esse cuidado, o que poderia ocorrer? Inconsistências seríssimas! Imagine que a versão Green cria colunas ou muda formatos que a Blue desconhece – dados inseridos pela Green podem quebrar a Blue assim que se tenta voltar atrás. De fato, sincronização de esquema e dados é apontada como um risco do Blue/Green: se não mantida, leva a **resultados imprevisíveis ou perdas de informação**. A solução passa por planejamento: usar feature toggles, migrações compatíveis e estratégias de duplicação de escrita/leitura (por exemplo, escrever nos dois formatos durante a transição). Tudo isso aumenta a complexidade do processo, mas é essencial para colher os benefícios do Blue/Green sem corromper dados.

ALGORITMOS DE PROMOÇÃO SEGURA E ROLLBACK DETERMINÍSTICO

Do ponto de vista algorítmico, um deploy Blue/Green pode ser visto como:

- **Preparação (pré-promoção):** implantar versão nova (Green) em modo isolado, rodar testes extensivos e sanidade. Garantir que o ambiente G está saudável e, idealmente, quente (recebendo tráfego interno ou sendo monitorado) antes da virada.
- **Cutover (promoção):** atualizar o roteador/balanceador de carga para enviar tráfego para Green. Isso deve ocorrer de forma **determinística e rápida**. Em Kubernetes, isso significa alterar um Service ou Ingress para apontar para os pods verdes. Em termos de algoritmo distribuído, o roteamento pode ser considerado uma variável global que precisamos

comutar de valor B para G; implementar via uma única fonte de verdade (como a configuração do load balancer) garante atomicidade lógica.

- **Validação pós-promoção:** monitorar de perto métricas e logs assim que Green assumir produção. Aqui entra a instrumentação: se algo estranho for detectado (erros aumentando, desempenho caindo), acionamos imediatamente o rollback.
- **Rollback (reversão):** consiste simplesmente em reconfigurar o roteador de volta para Blue. Note que isso é determinístico – não há ambiguidade sobre o que fazer, nem múltiplos passos: é uma operação única de swap, por isso extremamente rápida (pense em “apertar um botão vermelho de emergência”). Em contrapartida, em estratégias tradicionais, rollback significa desinstalar ou reimplantar uma versão anterior, o que envolve muito mais etapas e tempo. No Blue/Green, como destaca a documentação do Snyk, podemos redirecionar instantaneamente as requisições de volta para a versão antiga caso problemas surjam, sem downtime para os usuários.

Vale mencionar que alguns sistemas implementam variações onde o roteamento não muda 100% de uma vez – às vezes se faz um **cutover gradual** (tráfego canary dentro do Blue/Green). Mas aí já estamos hibridizando com estratégia de canary releases. O Blue/Green puro é “tudo ou nada”: primeiro Blue servindo 100%, depois Green servindo 100%.

Se optarmos por um incremento progressivo (por exemplo, 10% do tráfego para Green, depois 50%, etc.), estamos introduzindo etapas extras no algoritmo, o que complica a análise determinística do rollback. Entretanto, muitas ferramentas atuais (como **Argo Rollouts** no Kubernetes) suportam esse modo misto, permitindo validar a nova versão com parcela dos usuários antes do switch total.

ANÁLISE DE COMPLEXIDADE E CUSTO COMPUTACIONAL EM CLUSTERS GRANDES

O Blue/Green, por sua natureza, **duplica recursos** no momento da transição – dois ambientes completos rodam simultaneamente. Em termos de uso de CPU/RAM,

isso é um overhead de ~100% durante o deploy (ou continuamente, se mantivermos ambos os ativos sempre). Em clusters pequenos, talvez isso passe despercebido, mas em escala de milhares de pods e centenas de serviços, o impacto em custos de infraestrutura é real.

Empresas como a Etsy relataram manter 200% da capacidade (dobro de hosts) para suportar Blue/Green, o que é “caro e ineficiente” se perpetuado por muito tempo. Uma boa prática é desativar ou dimensionar para zero o ambiente antigo assim que o novo se estabilizar, reduzindo custos – ou até aproveitar o ambiente inativo para disaster recovery após replicar as mudanças (afinal, você pode mantê-lo pronto como contingência).

Quanto à complexidade computacional, do ponto de vista do control plane Kubernetes, a troca de um Service selector ou update de ingress é $O(1)$ para o desenvolvedor, mas o cluster realiza trabalho proporcional ao número de nós/pods: o kube-proxy em cada nó atualizará suas regras de roteamento para refletir o novo conjunto de endpoints.

Em um cluster grande (digamos 1000 nós, 10000 pods), essa propagação pode levar alguns segundos, mas ainda assim é bem controlada. A operação de swap em si tende a ser **limitada pela latência de rede e convergência do sistema**, não pelo tamanho do cluster em si – é por isso que falamos em “downtime zero ou próximo de zero”. Claro, se o cluster estiver saturado ou o etcd sob carga, atrasos podem aparecer.

Ainda assim, comparado a um rolling-update de 10000 pods (que levaria tempo linear no número de pods para concluir), o Blue/Green realiza um salto praticamente constante do ponto de vista de quem consome o serviço. **A escala impacta mais o custo do que a possibilidade**: já vimos casos de Blue/Green em clusters enormes de produção (big techs) onde a sincronização de milhares de endpoints ocorre de forma ordenada. Um risco em escala é a chamada “tempestade de conexões”: ao alternar todos os usuários de uma vez, a versão Green subitamente recebe toda carga.

Se houver caches frios ou JIT para esquentar, isso pode gerar um pico. Por isso, algumas arquiteturas optam por aquecer o ambiente Green com tráfego sombra ou gradativo antes do switch completo. Em resumo, em clusters grandes o Blue/Green

segue viável, mas requer monitorar esses efeitos colaterais (pico instantâneo de carga, latência de reconfiguração de rede, etc.) para assegurar que “zero downtime” não vire “pico de 2 segundos percebido” – ainda assim, bem melhor que minutos de indisponibilidade.

MODELAGEM MATEMÁTICA DE JANELAS DE CORTE E MINIMIZAÇÃO DE IMPACTO

Podemos modelar o risco de uma implantação como uma função do tempo em que uma versão potencialmente falha fica exposta. Suponha que a qualquer momento após o deploy, exista uma probabilidade p de detecção de um bug crítico, e que a taxa de requisições seja λ (req/s).

Se a nova versão Green falhar depois de Δt segundos de tráfego, aproximadamente $\lambda \cdot \Delta t$ requisições de usuários terão sido atendidas pela Green possivelmente com erro. O objetivo é minimizar esse número.

O Blue/Green atua justamente minimizando Δt , porque ao primeiro sinal de problema nós reversionamos instantaneamente para Blue (praticamente zerando o tempo extra de indisponibilidade percebida).

Além disso, não precisamos esperar janelas de baixo movimento: como não há interrupção formal do serviço, a troca pode ocorrer em horários normais sem prejudicar usuários, não dependemos mais daquele deploy às 3h da. Ainda assim, se quisermos ser ultra cautelosos, podemos planejar a virada em momentos de λ mais baixo (madrugada local, por exemplo), mas muitas empresas já não se preocupam com isso – preferem realizar em horário comercial, quando toda equipe está a postos para reagir, do que de madrugada com menos suporte técnico.

Em termos de funções de risco, digamos que definimos uma função de impacto $I(\Delta t) = \int_{t_0}^{t_0 + \Delta t} \lambda(t) dt$ (número de usuários afetados durante Δt). O Blue/Green tenta minimizar I tanto reduzindo Δt (via rollback rápido) quanto permitindo escolher um t_0 onde $\lambda(t)$ seja aceitável.

Em contrapartida, estratégias de deploy tradicionais sem rollback fácil podem incorrer em tempos de detecção e mitigação muito maiores (às vezes só se resolve reimplantando versão antiga, o que pode demorar vários minutos ou horas). Portanto,

matematicamente, Blue/Green domina outras estratégias no quesito minimizar área sob a curva de impacto – claro, assumindo que temos monitoramento e automação eficientes para detectar e reagir instantaneamente.

VERSIONAMENTO SEMÂNTICO E TOPOLOGIA DE DEPLOYMENTS ATIVOS/PASSIVOS

No contexto Blue/Green, é comum associar a versão Blue a algo como “v1.x” e Green “v2.0” (quando há uma grande mudança) ou simplesmente v1.1 vs v1.0. O importante não é o número em si, mas sim o **significado semântico**: versões que introduzem quebras de compatibilidade (major) requerem mais cautela.

Conforme as boas práticas de semver, incrementos major implicam que talvez não seja possível ter duas versões ativas simultaneamente sem adaptação – por isso que, ao fazer Blue/Green de um sistema 2.0 vs 1.0, deve-se garantir que 2.0 suporte operar em paralelo com 1.0 por um tempo. Já incrementos menores (minor, patch) idealmente mantêm compatibilidade, então o Blue/Green torna-se mais simples, praticamente garantido.

A estratégia Blue/Green convive muito bem com **versionamento semântico disciplinado**: se os contratos de dados e APIs forem evoluídos de forma aditiva ou compatível, podemos ter Blue e Green atendendo usuários distintos sem conflitos durante a transição. Por outro lado, se versão Green remove um campo que Blue ainda precisava (violando compatibilidade), a coexistência se torna problemática. Uma dica prática das empresas é integrar testes de compatibilidade no pipeline: sempre que uma mudança de versão for implantada no Green, testam-se cenários mistos (ex.: cliente antigo falando com serviço novo ou vice-versa, dependendo do contexto) para evitar surpresas.

Em termos de **topologia**, a estrutura Blue/Green é classicamente descrita como **ativo/passivo**: um ambiente serve produção (ativo) enquanto o outro fica em espera (passivo). Diferente de um cluster active-active (onde dois ambientes servem a mesma carga simultaneamente por motivos de redundância), aqui a duplicidade é usada para facilitar a troca, não para balancear carga.

Durante a transição, temporariamente ambos podem estar ativos (um recebendo tráfego novo, outro escoando conexões antigas ou servindo ainda parte dos usuários). Essa fase é curta. Algumas implementações, como vimos, preferem fazer um breve período de tráfego dividido (lembra do item 5. Traffic Shifting (Optional) que certas empresas aplicam).

Mas, conceitualmente, o Blue/Green é bipartido: o Blue ou o Green é o “Production” de fato em cada momento. A Etsy, por exemplo, apelidou seus ambientes Blue/Green de “flip” e “flop” – sempre um deles estava com tráfego, o outro ocioso e pronto para assumir. Essa metáfora lúdica (referência a circuitos eletrônicos flip-flop) ilustra bem a topologia: duas metades, um sistema biestável onde só um lado fica ligado ao mundo externo de cada vez.

CONTROLE DE RELEASES COM RBAC E GITOPS (ARGOCD/FLUX) E IDEMPOTÊNCIA DE MANIFESTS

Por fim, é importante conectar a estratégia Blue/Green à governança de mudanças e boas práticas de entrega contínua. Em equipes grandes, nem todos podem ou devem apertar o “botão” que faz a virada de tráfego – geralmente, definem-se permissões via **RBAC** (Role-Based Access Control) quem tem permissão para promover o Green a produção. Isso assegura que a troca seja coordenada e, de preferência, realizada através do pipeline oficial (CI/CD ou ferramentas GitOps) em vez de comandos manuais ad-hoc.

Ferramentas GitOps como **ArgoCD** ou **Flux** podem gerenciar Blue/Green de forma declarativa: você mantém no repositório as definições dos dois ambientes e, quando quer promover, faz um merge ou alteração nos manifests (por exemplo, alterando um Service ou a referência de imagem da Deploy para apontar para a versão nova).

O controlador GitOps aplica a mudança no cluster. Uma vantagem disso é a **auditabilidade** – cada promoção fica registrada como um commit, facilitando compliance e pós-mortem. Além disso, essas ferramentas costumam exigir configurações **idempotentes**: ou seja, aplicar o manifesto de configuração repetidamente não deve causar efeito adverso – ele sempre converge para o mesmo estado desejado.

O Kubernetes por si já promove esse modelo declarativo: o comando `kubectl apply` é idempotente, pode ser executado múltiplas vezes obtendo o mesmo resultado final. Isso é fundamental em Blue/Green, pois queremos poder reverter ou re-aplicar configurações sem medo de “correr um script de rollback duas vezes” e quebrar algo.

Provar formalmente a idempotência dos manifests significa garantir que, não importa quantas vezes executemos a sync do ArgoCD ou repliquemos os YAML, o cluster ficará no estado Blue ou Green pretendido, e nunca num estado intermediário inconsistente. Uma prática comum é testar o diff de configuração antes de aplicar – o ArgoCD, por exemplo, mostra se há drift. Se o manifesto de promoção já estiver aplicado, ele detecta “já está em Green” e não muda nada (ou seja, repetição do comando não causa problema).

Então, a filosofia GitOps combina bem com Blue/Green: define-se claramente os dois “modos” do sistema e alterna-se entre eles via versionamento de configuração, garantindo reproduzibilidade e controle de acesso. Inclusive, algumas equipes implementam aprovações manuais no fluxo GitOps para a etapa do switch – o pipeline prepara o Green, mas só aplica o corte quando alguém com autoridade aprova a PR. Isso adiciona uma camada de segurança e revisão humana, importante em sistemas críticos.

MERCADO, CASES E TENDÊNCIAS

A estratégia Blue/Green, embora conceitualmente simples, rende histórias interessantes no mundo real – algumas de sucesso notável, outras com lições aprendidas na dor. Vamos passear por alguns casos e tendências, numa conversa mais leve.

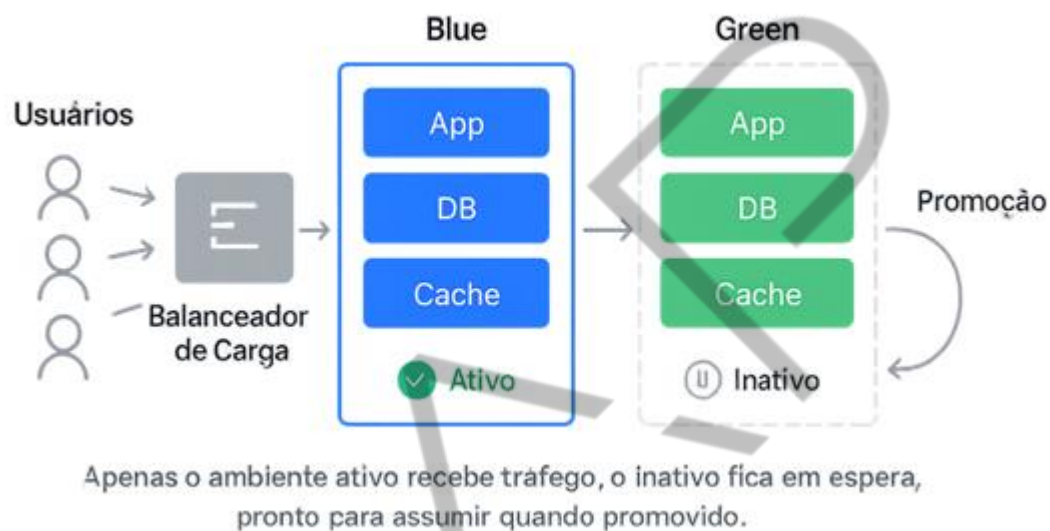


Figura 4 – Representação simplificada do conceito Blue/Green
Fonte: System Design But Simple (2025)

Começemos com um caso positivo: a **Etsy**, famoso marketplace de artesanatos, adotou Blue/Green para seu serviço de busca. Eles carinhosamente chamavam os ambientes de “flip” e “flop” – inspirados no circuito eletrônico bistável – e mantinham ambos atualizados em paralelo.

Enquanto um deles (“flip”) servia todas as buscas do site, o outro (“flop”) era atualizado com uma nova versão do mecanismo de busca. Assim que a equipe validava que tudo estava ok no flop, bastava “virar a chave” no código de configuração e pronto: o flop passava a receber todas as consultas dos usuários, tornando-se o novo ativo, e o flip ficava de reserva.

Esse processo permitiu à Etsy realizar até mudanças drásticas (o serviço de busca deles é cheio de estado e cache) com segurança. Sempre havia um refúgio: se algo desse muito errado, eles revertiam apontando de volta para o ambiente anterior em questão de segundos. Essa confiança mudou a cultura de deploy da empresa –

os desenvolvedores sabiam que podiam experimentar melhorias e, no pior dos cenários, daria para retornar à versão estável rapidamente.

Com o tempo, porém, perceberam um aspecto curioso: mudar 100% do tráfego de uma vez era seguro, mas **estressante**. Sem tráfego gradativo, não era possível detectar um problema de forma antecipada – ou tudo funcionava, ou tinham que voltar atrás. Por isso, a Etsy evoluiu para mesclar Blue/Green com **canary releases**, mandando um pequeno percentual de requisições para o novo lado antes do switch completo (eles até desenvolveram uma ferramenta interna chamada Switchboard para isso).

Essa tendência de deploys híbridos é vista em muitas empresas de ponta: usam Blue/Green como base (dois ambientes isolados), mas combinam com técnicas canary ou feature flags para uma promoção mais suave e observável.

Falando em feature flags, o mercado tem integrado muito o uso de flags com Blue/Green. Ferramentas como LaunchDarkly popularizaram a ideia de decouple deploy de release – ou seja, você faz o Blue/Green, mas deixa certas funcionalidades novas desligadas por flag até ter certeza que tudo está bem, e só então liga as features para os usuários.

Isso dá um controle granular: se detectar um bug numa funcionalidade específica, pode desligar só aquele flag (desabilitando a feature) em vez de reverter todo o deploy. Assim, Blue/Green + flags dá quase um “rollback parcial” de comportamentos, algo muito útil. Essa abordagem é forte, especialmente em produtos web que sofrem A/B testing: você pode direcionar uma parcela dos usuários para Green e ainda dentro do Green habilitar ou não certos recursos via flags, comparando métricas.

Do lado das **cautelas e fracassos**, há histórias de terror também (que depois geram melhorias). Um cenário comum: uma empresa tenta aplicar Blue/Green em todas as suas aplicações ao mesmo tempo, sincronizadamente. Imagine um grande banco com dezenas de sistemas interconectados – eles pensam: “Vamos fazer Blue/Green global, atualizar tudo junto, Blue inteiro para Green inteiro”. A intenção de evitar incompatibilidades entre sistemas é boa, mas na prática isso se torna um pesadelo logístico.

Coordenar múltiplas aplicações diferentes para trocar de cor simultaneamente introduz dependências que eliminam a vantagem da simplicidade. Uma empresa (que preferimos não nomear, mas você adivinha: aquelas monolíticas financeiras) sofreu semanas tentando sincronizar deploys interdependentes – descobriu da pior forma que o ideal é **desacoplar e versionar contratos**, não tentar orquestrar big-bang azuis e verdes de tudo ao mesmo tempo.

Após esse susto, eles reformularam a arquitetura para microserviços mais independentes e adotaram compatibilidade retroativa entre sistemas: cada sistema faz Blue/Green no seu ritmo, garantindo que suas interfaces aceitem tanto a versão velha quanto a nova dos colegas por um período. A lição? Blue/Green não elimina a necessidade de arquitetura bem pensada; pelo contrário, ele evidencia onde há acoplamentos excessivos.

Outra anedota envolve **custo e cultura**: uma startup tentou implementar Blue/Green em sua pequena infraestrutura e quase faliu no processo. Manter duas cópias de tudo no ar dobrava a conta de cloud, o que para eles era pesado. Além disso, os desenvolvedores não estavam acostumados com deploys contínuos e acabavam acumulando mudanças de funcionalidade esperando um “momento certo” para mandar para o ambiente Green.

Ou seja, não faziam deploy contínuo no ambiente passivo – tinham medo de enviar código demais sem liberar, e acabavam perdendo o benefício do feedback rápido. Resultado: quando finalmente trocavam Blue->Green, era um salto gigantesco de versão, muito arriscado. Esse antipadrão foi discutido por especialistas: se você não **usa continuamente o ambiente inativo**, corre o risco de transformá-lo num segundo estágio de QA gigante, e não num clone de produção realmente testado.

A empresa em questão ajustou o rumo: passou a fazer deploy no Green a cada mudança pequena (mesmo que não fosse trocar já), mantendo-o atualizado, e só acionava a troca quando acumulava confiança. Isso sim é Blue/Green de verdade: tratar o “verde” quase como produção o tempo todo, apenas sem tráfego de usuários até decisão de promovê-lo.

No panorama atual, vemos a ascensão de ferramentas que automatizam Blue/Green. No ecossistema Kubernetes, o **Argo Rollouts** (um controlador de deploys customizado) suporta Blue/Green nativamente – você define um Rollout com

estratégia blueGreen e ele gerencia os ReplicaSets e Services necessários, até permitindo preview (deixar o Green pronto e esperar um manual promotion) e analysis (integrado com métricas para decidir se promove ou reverte automaticamente).

O **Flagger** (do Flux project) é outra solução que integra com service mesh/Istio para gerenciar o tráfego entre versões azul e verde de forma inteligente. Essas ferramentas indicam uma tendência de **automação declarativa**: em vez de scripts manuais de patch (como fizemos no Hands-on), configura-se políticas, e o controlador realiza o Blue/Green conforme critérios. Isso é particularmente útil em ambientes de larga escala e equipes com maturidade em SRE, pois podem incorporar checagens de saúde e rollback automático sem intervenção humana.

Por fim, não podemos deixar de citar terminologias e variações divertidas: você já ouviu falar de **deploy Red/Black**? É a mesma coisa que Blue/Green, só que popularizada pela Netflix/Spinnaker com nomes de cores diferentes (talvez "red" soe mais sério que "blue"). O LaunchDarkly até brinca que é um "nome mais dramático" para enfatizar o corte imediato.

E **Rainbow Deployments**? Esse termo informal refere-se a manter múltiplas versões ativas simultaneamente – por exemplo, um SaaS manter versão Gold e Silver do app rodando para diferentes clientes. Não é exatamente Blue/Green, mas mostra como a ideia de multi-versão em produção está evoluindo. Poucas empresas fazem isso, pois a complexidade cresce (suporte a N versões ativas), mas em cenários de transição longa ou beta testing prolongado com usuários reais, pode acontecer – daí o "arco-íris" de versões.

Em resumo, no campo de batalha do TI, a estratégia Blue/Green tem se provado uma poderosa aliada para quem precisa lançar rápido sem cair no abismo do downtime. Empresas que a adotaram relatam maior confiança para inovar e um sono mais tranquilo dos engenheiros de plantão. Mas também aprenderam a duras penas a respeitar os cuidados necessários – especialmente com bancos de dados e com a disciplina de manter tudo orquestrado. A tendência é que Blue/Green não vá a lugar nenhum; pelo contrário, com a cultura DevOps e ferramentas GitOps em alta, veremos cada vez mais **deploys invisíveis** aos usuários e reversões automáticas evitando incidentes.

E você, já se imagina apertando um botão e trocando toda sua produção de cor? Parece mágica, mas é apenas boa engenharia!

EMANIP

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos a fundo a estratégia de **deploy Blue/Green** e seus desdobramentos. Você aprendeu como esse método permite operar dois ambientes paralelos para alternar versões de uma aplicação sem interrupções perceptíveis. Na introdução, discutimos os desafios de lançar novas versões em sistemas críticos e entendemos o valor do Blue/Green em mitigar riscos e eliminar downtime. No Hands On, vimos na prática (com direito a código Rust!) como configurar pods azuis e verdes em um cluster Kubernetes e utilizar Services para comutar tráfego de forma instantânea – reforçando a ideia de rollback rápido e fácil.

Em seguida, mergulhamos em conceitos avançados na seção **Saiba Mais**: formalizamos o problema do cutover seguro, refletimos sobre consistência de dados e compatibilidade de esquemas (especialmente no contexto de bancos de dados), e analisamos algoritmos, complexidade e até uma modelagem de risco para implantações Blue/Green. Também relacionamos a estratégia com versionamento semântico e vimos como controlar tudo via GitOps com segurança e idempotência.

Você sai desta aula entendendo não apenas o que é Blue/Green, mas porque ele importa e como aplicá-lo com maestria e segurança em ambientes reais. Leve esse conhecimento adiante e experimente aplicá-lo em seus projetos – e lembre-se: deploys sem downtime não são magia, e sim resultado de planejamento, automação e boas práticas que agora estão no seu repertório!

REFERÊNCIAS

FOWLER, M. **Blue Green Deployment**. martinfowler.com. 2010. Disponível em: <https://martinfowler.com/bliki/BlueGreenDeployment.html>. Acesso em: 8 set. 2025.

KILLEEN, S. **Avoid the Blue/Green Deployment Blues: Watch out for these 10 Pitfalls**. seankilleen.com. 2020. Disponível em: <https://seankilleen.com/blue-green-deployment-pitfalls>. Acesso em: 8 set. 2025.

KING, R. **Improving the Deployment Experience of a Ten-Year Old Application**. Code as Craft. 2021. Disponível em: <https://codeascraft.com/deployment-experience>. Acesso em: 8 set. 2025.

LAUNCHDARKLY (Blog). **Blue-Green vs. Rolling Deployments: Pros, Cons & Implementation**. 2023. Disponível em: <https://launchdarkly.com/blog/blue-green-vs-rolling>. Acesso em: 8 set. 2025.

MOREAU, S. **Blue-Green Deployment in 1 diagram and 195 words**. System Design, but Simple (Substack). 2025. Disponível em: <https://systemdesign.substack.com/p/blue-green-deployment>. Acesso em: 8 set. 2025.

NAVEED, M. **Blue-Green Deployments with Kubernetes: A Comprehensive Guide**. Medium. 2023. Disponível em: <https://medium.com/cloud-native-daily/blue-green-deployments>. Acesso em: 8 set. 2025.

OCTOPUS DEPLOY. **Blue/green deployments: how they work, pros and cons, and 8 critical best practices**. 2025. Disponível em: <https://octopus.com/blog/blue-green-deployments>. Acesso em: 8 set. 2025.

SMITH, J. **Blue-green deployment strategy explained**. Snyk Blog. [s.d.]. Disponível em: <https://snyk.io/blog/blue-green-deployment-explained>. Acesso em: 8 set. 2025.

THE Difference Between Rolling and Blue-Green Deployments. **Harness Engineering Blog**. 2024. Disponível em: <https://harness.io/blog/rolling-vs-blue-green-deployments>. Acesso em: 8 set. 2025.

PALAVRAS-CHAVE

Kubernetes. Rollout. Canary.

EMPRE

The background is a dark teal color with a network of thin, light teal lines and small circular nodes. Some nodes are larger and more prominent, while others are smaller and less visible. The lines connect the nodes in a complex, web-like pattern, suggesting a digital or technological theme.

POSTECH